

RoboMonitor: Label-Efficient Runtime Monitoring of Robot Task Execution via Predictive Representation Learning

Abhiroop Ajith^{1,2}, Gokul Narayanan^{1,†}, Kyle Coelho¹, Tingji Zhao¹, Yash Shahapurkar¹, Brian Zhu¹, Melih Erdogan¹, Ted Krubasik¹, Constantinos Chamzas², and Eugen Solowjow¹

Abstract—Learned robot policies produce actions, but their outputs alone do not establish whether execution is progressing as intended. Robot execution monitoring requires identifying the current execution phase, detecting failures, and recognizing task completion from observations available during execution. Training such monitors requires annotations that are scarce in datasets collected for robot-policy learning. We present RoboMonitor, a label-efficient vision–language execution monitor that learns from these datasets before introducing monitoring supervision. We pre-train on 25 hours of multi-camera trajectories spanning 12 manipulation tasks and two robot embodiments, using action-conditioned future-feature prediction, inverse dynamics, and masked-present prediction. We then transfer the learned visual and context encoders to a causal monitor and apply temporal supervised fine-tuning (Temporal SFT), which combines supervision throughout each observation window with consistency objectives within and across overlapping windows. At deployment, RoboMonitor requires only the task instruction and camera observations.

On a four-task monitoring benchmark, RoboMonitor trained with 52 labeled episodes achieves 93.1% mean phase accuracy and 85.9% macro recall over two fine-tuning seeds, exceeding Qwen3-VL and Robometer trained with the same monitoring supervision. Its phase accuracy also exceeds that of both Qwen3-VL and Robometer trained with 100 episodes. A Qwen3-VL ablation shows that Temporal SFT reduces mean spurious phase switching from 15.23% to 4.95%. In closed-loop deployment, the integrated system completes 39 of 40 simulated Toolbox Sorting trials and 35 of 40 real-world Reel Packing trials, with no false recovery triggers observed.

I. INTRODUCTION

Learned robot policies are increasingly used for long-horizon manipulation [1]–[3]. They can execute complete tasks or serve as individual skills within an orchestration system that combines learned and conventional robot controllers [4]–[6]. In either setting, execution requires feedback about the physical outcome of the robot’s actions. A grasp may miss, an object may be dropped, or a skill may terminate before achieving its intended effect. An execution monitor provides this feedback by identifying the current phase, detecting failures, and recognizing task completion from the task instruction and recent observations. These predictions can inform task progression, recovery, replanning, or human intervention. Their temporal stability also matters: phase estimates that fluctuate while the robot remains in the

same execution stage can give an orchestrator inconsistent information about task progress.

Training such a monitor requires dense temporal annotations that are rarely available in robot-learning datasets. Policy datasets contain large amounts of synchronized video, robot state, and action data, but are dominated by nominal executions and generally do not indicate the current phase, the onset of a failure, or when the overall task has completed. Collecting representative failures and annotating trajectories frame by frame is costly. However, the large amount of robot data already collected during policy development provides an opportunity to learn relevant representations of the task before monitoring annotations are introduced.

We introduce **RoboMonitor** (Figure 1), a two-stage approach to label-efficient execution monitoring. First, predictive pre-training adapts the visual encoder of a vision–language model (VLM) and learns a temporal context encoder from multi-camera robot trajectories. The training objective combines future-feature prediction conditioned on recorded robot state and commands with inverse dynamics and masked-present prediction. We then transfer the visual and context encoders to a causal monitor and fine-tune it on trajectories annotated with execution phase, failure, and task completion. Our Temporal SFT procedure supervises the monitor’s prediction at every timestep of each observation window rather than only at the final one, and encourages consistent predictions within and across overlapping windows. Robot state and commands are confined to pre-training; the deployed monitor uses the task instruction and camera observations.

Our contributions are: (i) a predictive pre-training and representation transfer approach that learns from robot trajectories without monitoring annotations; (ii) a causal execution monitor trained with dense supervision and temporal consistency objectives; and (iii) an evaluation of monitoring accuracy and stability at two annotation budgets, together with closed-loop execution and recovery in simulation and on a real robot.

II. RELATED WORK

Runtime Failure Detection. Recent work has explored failure detection using both VLM-based verification and signals from the underlying control policy. AHA [7] detects and reasons about failures for a specified current subtask, while Guardian [8] verifies whether a given subtask was successfully executed from before-and-after observations.

¹The authors are with Siemens Corporation, USA. This work was conducted while Abhiroop was an intern at Siemens Corporation.

²Abhiroop and Constantinos are with the ELPIS Lab, Worcester Polytechnic Institute, USA.

[†]Corresponding author:
gokul.sathya.narayanan@siemens.com

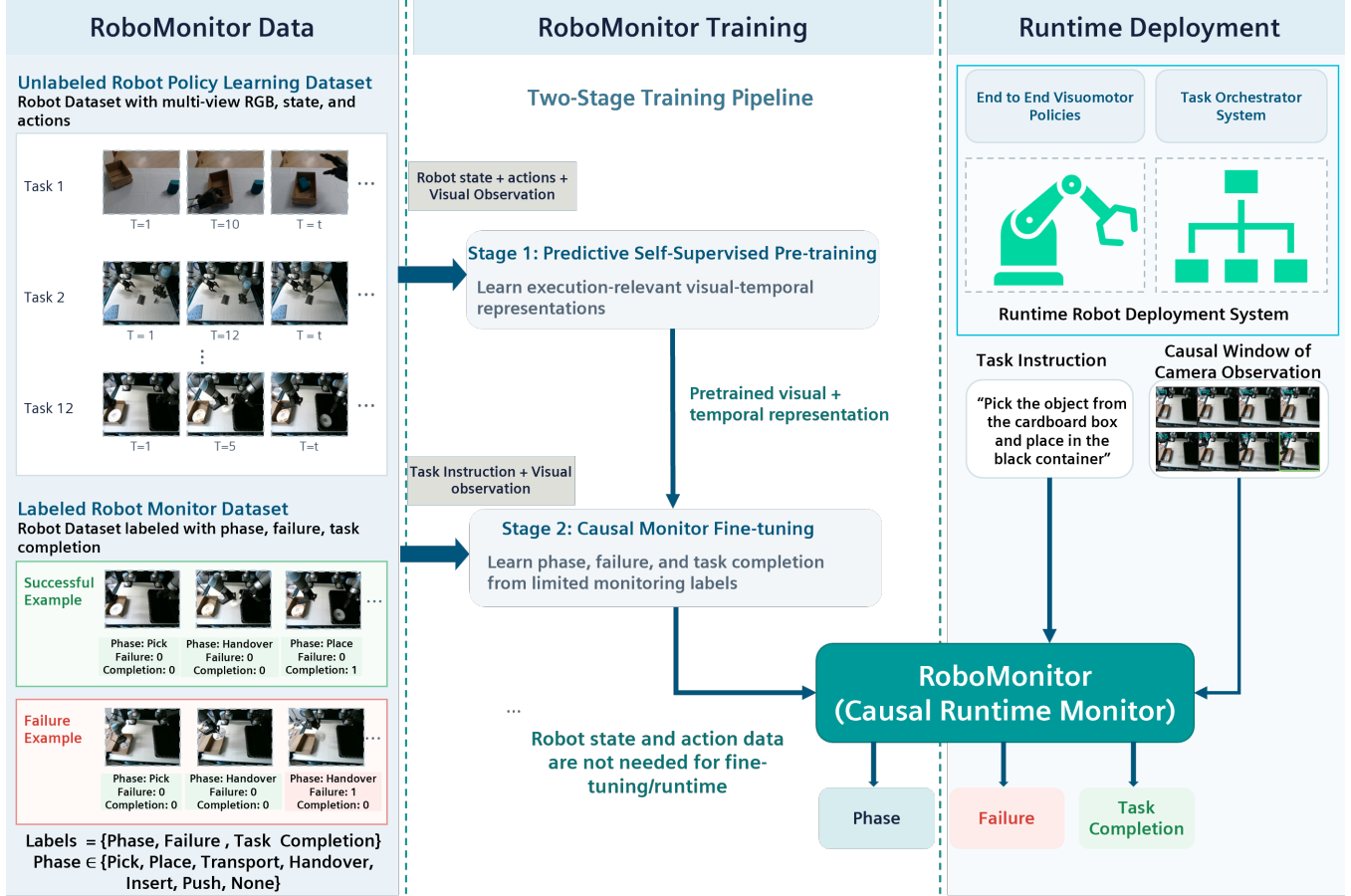


Fig. 1. **Overview of RoboMonitor.** Training has two stages: (1) predictive self-supervised pre-training on unlabeled robot trajectories, which uses multi-view observations together with recorded robot state and executed actions to learn execution-relevant visual and temporal representations; and (2) causal supervised fine-tuning (Temporal SFT) on a smaller monitoring-labeled dataset to predict execution phase, failure, and task completion. At deployment, RoboMonitor receives only the task instruction and camera observations, so it can monitor both end-to-end visuomotor policies and task-orchestration systems without access to the underlying controller.

These works primarily perform subtask-conditioned verification, whereas RoboMonitor continuously infers the current execution phase, failure state, and task completion directly from causal visual observations without requiring the current subtask to be specified. Other approaches [9]–[13] detect failures from probing the control policy’s representations, action outputs, or comparing reference trajectories; unlike these methods, RoboMonitor is decoupled from the underlying control policy at deployment.

Reward Models and Task Progress Estimation. Reward models provide dense signals about task execution. Robometer [14] learns general-purpose progress and success rewards from expert, suboptimal, and failed trajectories. SARM2 [15] estimates the current action-primitive stage to condition a multi-task value model for policy self-improvement, while ROVER [16] uses recursive VLM reasoning to estimate frame-level task progress over long-horizon executions.

These methods primarily produce reward, progress, or reasoning signals for policy evaluation and improvement. In contrast, RoboMonitor treats execution monitoring as the primary objective, jointly predicting execution phase, failure, and task completion from causal visual observations, with explicit consideration of temporal stability and online inference.

Visual and sensorimotor pre-training. Self-supervised pre-training has produced reusable representations for robot learning. R3M learns visual features from human video [17], VIP learns visual rewards through value-based pre-training [18], and RPT jointly models robot images, proprioception, and actions before transfer to control policies [19]. Joint-embedding predictive architectures learn by predicting target representations rather than reconstructing pixels [20], [21]. We use this predictive principle to learn from multi-camera robot trajectories and transfer both the

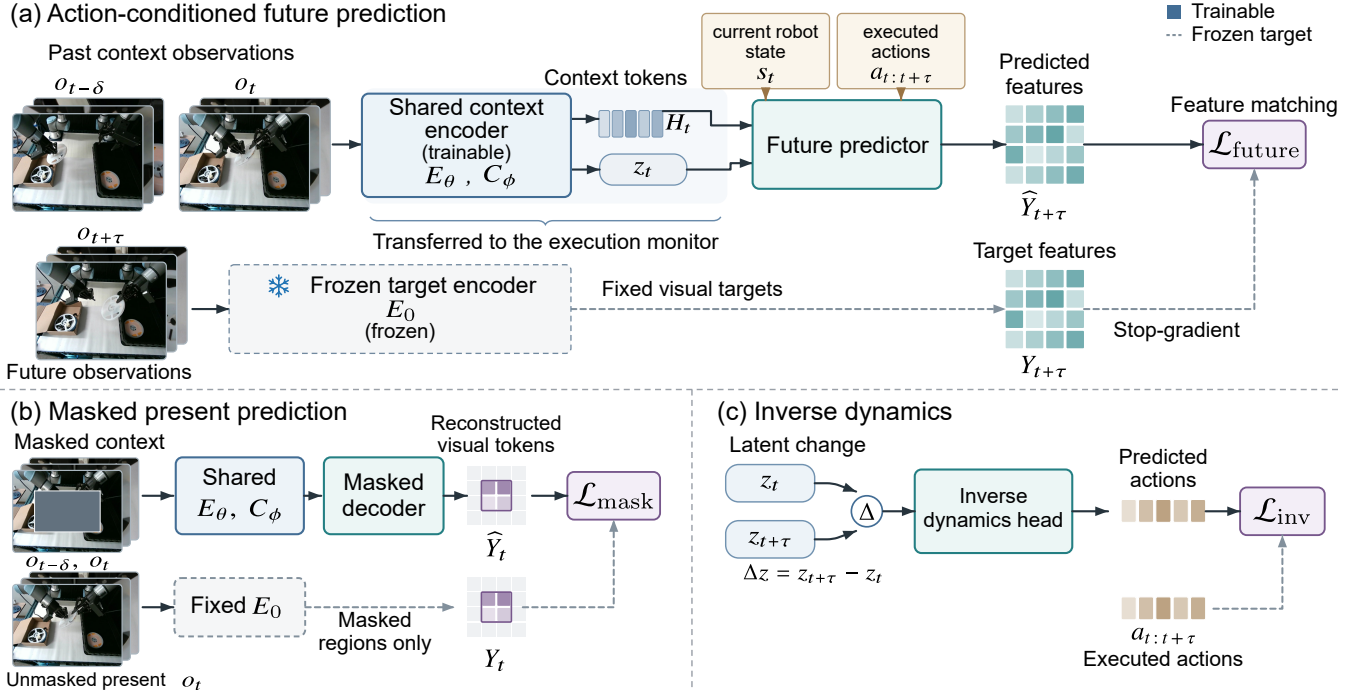


Fig. 2. **Predictive robot pre-training (stage 1).** (a) Visual and context encoders process recent multi-camera observations; a predictor uses their features, recorded robot state, and commands to estimate future visual features. A fixed copy of the original visual encoder supplies the targets. (b) Masked-present prediction estimates features at masked spatial locations. (c) Inverse dynamics predicts commands from the difference between context summaries. The adapted visual and context encoders are retained for execution monitoring.

adapted visual encoder and its temporal context representation to runtime execution monitoring.

III. PROBLEM FORMULATION

We consider a manipulation task specified by an instruction g , executed either by an end-to-end policy or by a high-level planner that selects low-level skills from a library $\mathcal{K}$. At time t , the robot is observed through C cameras, $o_t = (o_t^1, \dots, o_t^C)$. Given the instruction and a causal observation window $w_t = (o_{t-H+1}, \dots, o_t)$, the execution monitor predicts

$$M_\theta(g, w_t) = (\hat{y}_t^{\text{phase}}, p_t^{\text{fail}}, p_t^{\text{comp}}). \quad (1)$$

where the outputs denote the current execution phase and the probabilities of failure and task completion.

We assume access to a large corpus of robot trajectories containing video, state, and actions, together with a smaller corpus carrying monitoring annotations. Our objective is to use the unlabeled trajectories to reduce the annotations required to learn M_θ . At deployment, the monitor receives only the task instruction and camera observations.

IV. METHODOLOGY

RoboMonitor is trained in two stages. Predictive pre-training (Figure 2) learns visual and temporal representations from recorded observations, robot state, and commands without monitoring annotations. Temporal SFT (Figure 3) transfers these representations to an execution monitor and trains its phase, failure, and completion predictions using causal observation windows.

A. Predictive Robot Pre-training

Action-conditioned future prediction. Let $o_t = (o_t^1, \dots, o_t^C)$ denote the synchronized observations from C cameras, s_t the recorded robot state, and $a_{t:t+\tau}$ the commanded action sequence over $[t, t + \tau]$, where τ is the prediction horizon. We initialize a visual encoder E_θ from Qwen3-VL-4B-Instruct [22] and adapt it using visual LoRA modules [23]. A separate copy of the original encoder, E_0 , provides prediction targets and remains fixed throughout pre-training. Following joint-embedding predictive learning [20], [21], we predict features of future observations rather than reconstructing their pixels.

We encode each available camera view at $t - \delta$ and t , where δ is a fixed context interval. The context encoder C_ϕ combines these visual tokens with camera, spatial-position, and relative-time information:

$$(H_t, z_t) = C_\phi(E_\theta(o_{t-\delta}), E_\theta(o_t)). \quad (2)$$

Here, H_t is the contextual token sequence and z_t is a summary computed from these tokens. Before future prediction, a learned projection of z_t is added to H_t , so the future-prediction loss also trains the summary projection.

The same observed scene can be followed by different robot commands. We therefore condition the future predictor P_ψ on the recorded state and intervening commands:

$$\begin{aligned} \hat{Y}_{t+\tau} &= P_\psi(H_t, z_t, s_t, a_{t:t+\tau}, \tau), \\ Y_{t+\tau} &= E_0(o_{t+\tau}). \end{aligned} \quad (3)$$

Prediction queries specify the camera and spatial location of each target token. Robot state and commands enter only the predictor, not E_θ or C_ϕ . The context features can therefore be computed from camera observations alone during monitoring.

For corresponding predicted and target tokens $\hat{y}$ and y , we use

$$\ell_{\text{feat}}(\hat{y}, y) = \frac{1}{d} \|\text{LN}(\hat{y}) - \text{LN}(y)\|_1, \quad (4)$$

where d is the token dimension and LN denotes layer normalization without learned affine parameters. Averaging this loss over valid spatial tokens, target cameras, and training examples gives $\mathcal{L}_{\text{future}}$.

Inverse dynamics. The forward predictor receives commands as input. We additionally train an inverse-dynamics head to predict those commands from the difference between two visual context summaries [24], [25]. Using the same encoders, we compute $z_{t+\tau}$ from observations at $t + \tau - \delta$ and $t + \tau$, so both contexts have duration δ . The inverse prediction is

$$\hat{a}_{t:t+\tau} = I_\omega(z_{t+\tau} - z_t, \tau). \quad (5)$$

Unlike the forward predictor, the inverse head receives neither the recorded state nor commands. We minimize a Huber loss over valid command entries, $\mathcal{L}_{\text{inv}}$, giving the summary difference an explicit action-prediction target. The later observations used for this objective are not inputs to the forward predictor.

Masked-present prediction. The future objective predicts features at $t + \tau$. We also introduce a prediction target at the present timestep t . In a separate pass, contiguous spatial regions are masked before the visual encoder’s attention layers. An auxiliary decoder uses the remaining visual context to predict features from E_θ applied to the unmasked present observation. We apply Equation 4 only at the masked target locations, obtaining $\mathcal{L}_{\text{mask}}$. This decoder receives neither robot state nor commands.

The complete pre-training objective is

$$\mathcal{L}_{\text{pre}} = \mathcal{L}_{\text{future}} + \lambda_{\text{inv}} \mathcal{L}_{\text{inv}} + \lambda_{\text{mask}} \mathcal{L}_{\text{mask}}, \quad (6)$$

where the coefficients weight the auxiliary losses. After pre-training, we retain E_θ and C_ϕ for execution monitoring. The target encoder, future predictor, inverse-dynamics head, and masked-present decoder are not used by the deployed monitor.

B. Causal Monitor Fine-Tuning

We initialize the VLM’s visual encoder with the pre-trained E_θ and train the execution monitor on phase, failure, and completion annotations. We use *Temporal SFT* to denote the supervision scheme: labeled causal readouts are trained throughout each observation window, with additional consistency objectives within and across overlapping windows.

Causal readouts. The monitor receives the task instruction g and an ordered window $w_t = (o_{t-H+1}, \dots, o_t)$ of H multi-camera observations. After the image tokens at each timestep j , we insert a learned $\langle |\text{state_reward}| \rangle$ token. This

readout token provides a dedicated position for extracting features for downstream prediction [26]. We call its VLM hidden state the *readout* r_j , which is used by the monitoring heads after feature fusion. Causal attention restricts r_j to the instruction and observations up to timestep j .

Although deployment uses only the final readout, causal attention makes every labeled position available for supervision without access to later observations. We therefore train on all labeled readouts in each window, rather than only the final one. The final position receives full weight, while earlier phase and failure predictions receive a smaller context weight. Positive failure examples receive additional weight because they are rare.

Transferring the predictive representation. The VLM readout combines the task instruction with the observation history. We augment it with features from the pre-trained context encoder C_ϕ . A separate frozen copy of E_θ supplies visual features to the frozen C_ϕ , preserving this branch while the VLM’s visual encoder is adapted through LoRA. For each supervised timestep j , the context branch processes a recent observation pair ending at j and produces the corresponding tokens H_j and summary z_j .

We form two fused features, indexed by $h \in \{\text{prog}, \text{fail}\}$:

$$\tilde{r}_j^h = r_j + A_h(z_j) + R_h(r_j, H_j), \quad (7)$$

where A_h projects the summary into the VLM feature space and R_h attends to the contextual tokens using r_j as the query. The residual output projections are initialized to zero, so initially $\tilde{r}_j^h = r_j$. Phase and completion share the feature indexed by prog; failure uses separate adapters. The output mappings are

$$\begin{aligned} p_j^{\text{phase}} &= \text{softmax}(F_{\text{phase}}(\tilde{r}_j^{\text{prog}})), \\ p_j^{\text{comp}} &= \text{sigmoid}(F_{\text{comp}}(\tilde{r}_j^{\text{prog}})), \\ p_j^{\text{fail}} &= \text{sigmoid}(F_{\text{fail}}(\tilde{r}_j^{\text{fail}})). \end{aligned} \quad (8)$$

The supervised objective combines phase cross-entropy with binary cross-entropy for failure and completion:

$$\begin{aligned} \mathcal{L}_{\text{sup}} &= \lambda_{\text{phase}} \mathcal{L}_{\text{phase}} + \lambda_{\text{fail}} \mathcal{L}_{\text{fail}} \\ &\quad + \lambda_{\text{comp}} \mathcal{L}_{\text{comp}}. \end{aligned} \quad (9)$$

Each loss incorporates the position and class weighting described above.

Temporal consistency. The supervised losses compare each prediction with its annotation, but do not directly penalize disagreement between predictions. We add two terms to address different forms of temporal inconsistency.

Within a window, we discourage short-lived phase changes using the truncated log-probability smoothing loss of MS-TCN [27]:

$$\begin{aligned} \Delta_{j,c} &= \log p_{j,c}^{\text{phase}} - \log p_{j-1,c}^{\text{phase}}, \\ \mathcal{L}_{\text{within}} &= \frac{1}{|\mathcal{A}|K} \sum_{j \in \mathcal{A}} \sum_{c=1}^K \min(\Delta_{j,c}^2, \kappa^2), \end{aligned} \quad (10)$$

where K is the number of phases, $\mathcal{A}$ contains positions for which both adjacent timesteps have phase labels, and κ is the

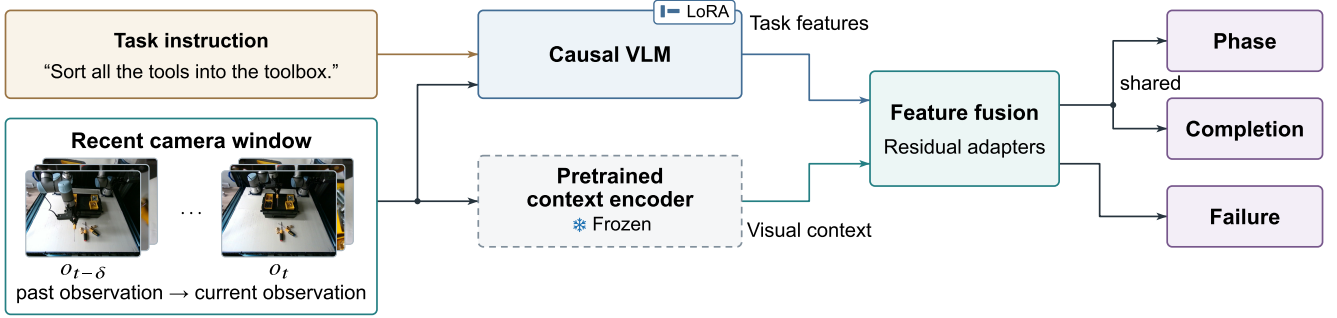


Fig. 3. **Causal execution monitor (stage 2).** The VLM processes the task instruction and a multi-camera observation window, producing a causal readout at each timestep. A separate frozen visual/context branch supplies features from the corresponding recent observation pair. Residual adapters combine these features using Equation 7. Phase and completion share fusion adapters; failure uses separate adapters. Deployment uses the predictions at the final timestep.

truncation threshold. Truncation caps the penalty on large changes rather than penalizing them increasingly as their magnitude grows.

Overlapping windows can produce different predictions for the same observation because they provide different preceding histories. We penalize this disagreement at shared timestamps. For paired windows a and b from the same episode, let $\mathcal{O}$ contain their shared observation timestamps. Superscripts a and b identify the window used to compute each prediction. We define

$$\mathcal{L}_{\text{overlap}} = \frac{1}{|\mathcal{O}|} \sum_{u \in \mathcal{O}} \left[\frac{1}{2} D_{\text{KL}}(\mathbf{p}_u^{a, \text{phase}} \| \mathbf{p}_u^{b, \text{phase}}) + \frac{1}{2} D_{\text{KL}}(\mathbf{p}_u^{b, \text{phase}} \| \mathbf{p}_u^{a, \text{phase}}) + \beta (p_u^{a, \text{fail}} - p_u^{b, \text{fail}})^2 \right], \quad (11)$$

where D_{KL} is Kullback–Leibler divergence and β weights the failure term. Symmetric KL compares the phase distributions, while squared error compares the failure probabilities.

The complete fine-tuning objective is

$$\mathcal{L}_{\text{mon}} = \mathcal{L}_{\text{sup}} + \lambda_{\text{within}} \mathcal{L}_{\text{within}} + \lambda_{\text{overlap}} \mathcal{L}_{\text{overlap}}. \quad (12)$$

We update the monitoring heads, fusion modules, and visual and language LoRA parameters; the remaining VLM parameters and the separate visual/context encoders stay fixed. At deployment, the monitor returns the phase, failure, and completion predictions for the final timestep, using only the instruction and camera observations.

V. EXPERIMENTS

We evaluate three questions:

Q1 (Label efficiency): Does RoboMonitor improve execution monitoring when labeled data are limited?

Q2 (Temporal stability): Does Temporal SFT reduce spurious phase switching compared with final-readout-only supervision?

Q3 (Closed-loop execution): Can RoboMonitor support closed-loop task execution and failure recovery within a task-orchestration system?

A. Data and Tasks

Self-supervised corpus. We pre-train RoboMonitor on 25 hours of multi-camera robot trajectories collected during policy development, spanning 12 manipulation tasks and two robot embodiments. Pre-training uses synchronized RGB observations, recorded robot state, and commanded actions, without phase, failure, or task-completion annotations.

Supervised monitoring tasks. We train a shared execution monitor on four manipulation tasks:

Reel Packing. One arm inserts its gripper into a reel to pick it up and transfers the reel to the other arm, which places it in a black tote. The annotated phases are PICK, HANDOVER, and PLACE.

Electronic Component Insertion. The robot picks up the electronic component, transports it, and inserts it into its testing module. The annotated phases are PICK, TRANSPORT, and INSERT.

Toolbox Sorting. The robot sorts tools and small parts, including screwdrivers, nuts, and bolts, into designated areas in Isaac Sim. The annotated phases are NONE, PICK, TRANSPORT, and PLACE.

Humanoid Grasping. The humanoid repositions a bin by pushing it to accommodate the arms’ limited reach when transferring an object between them. The annotated phases are PUSH, PICK, and PLACE.

Reel Packing and Electronic Component Insertion are represented in the self-supervised corpus, whereas Toolbox Sorting and Humanoid Grasping are absent. All four tasks are included during supervised fine-tuning. The monitor uses the union of their phase labels as a shared seven-class vocabulary, with separate binary outputs for failure and task completion.

B. Offline Evaluation

We compare supervised training budgets of 52 and 100 labeled episodes across the four tasks. The 100-episode set contains 27 Reel Packing, 26 Humanoid Grasping, 25 Toolbox Sorting, and 22 Electronic Component Insertion episodes. The 52-episode set contains 13 episodes per task and is nested within the larger set.

TABLE I

OFFLINE EXECUTION-MONITORING PERFORMANCE. ALL VALUES ARE PERCENTAGES; HIGHER IS BETTER EXCEPT SPURIOUS-SWITCH RATE ($\downarrow$). FINE-TUNED RESULTS ARE MEAN $\pm$ STANDARD DEVIATION OVER TWO SFT SEEDS. BOLD DENOTES THE BEST MEAN WITHIN EACH ANNOTATION BUDGET. POINTWISE SFT IS EVALUATED ONLY AT 52 EPISODES. GEMINI USES FEW-SHOT EXAMPLES WITHOUT PARAMETER UPDATES.

SFT episodes	Method	Phase		Completion	Failure	Spurious
		Accuracy	Macro Recall	AP	AP	Switch Rate $\downarrow$
52	RoboMonitor-4B (Ours)	93.09 $\pm$ 0.29	85.85 $\pm$ 0.41	87.60 $\pm$ 0.01	45.87 $\pm$ 0.68	5.58 $\pm$ 0.22
	Robometer-4B + Temporal SFT	88.77 $\pm$ 1.06	81.78 $\pm$ 0.41	85.42 $\pm$ 0.31	59.39 $\pm$ 3.11	5.27 $\pm$ 0.40
	Qwen3-VL-4B + Temporal SFT	86.76 $\pm$ 2.29	74.23 $\pm$ 5.69	74.01 $\pm$ 10.85	46.37 $\pm$ 6.99	4.95 $\pm$ 0.12
	Qwen3-VL-4B + Pointwise SFT	85.42 $\pm$ 0.58	73.93 $\pm$ 0.99	70.14 $\pm$ 2.29	30.34 $\pm$ 1.16	15.23 $\pm$ 7.30
100	RoboMonitor-4B (Ours)	93.63 $\pm$ 1.04	88.55 $\pm$ 0.73	92.29 $\pm$ 1.17	72.01 $\pm$ 0.14	3.17 $\pm$ 0.04
	Robometer-4B + Temporal SFT	91.65 $\pm$ 0.02	86.22 $\pm$ 0.12	90.98 $\pm$ 0.01	67.68 $\pm$ 0.43	3.76 $\pm$ 0.27
	Qwen3-VL-4B + Temporal SFT	91.19 $\pm$ 0.01	82.73 $\pm$ 0.64	84.81 $\pm$ 0.27	60.96 $\pm$ 0.05	3.58 $\pm$ 0.16
–	Gemini Robotics ER 2 (few-shot)	74.67	61.19	64.07	32.53	17.62

All methods are evaluated on the same held-out benchmark, comprising 66 episodes and 7,022 observation windows. We construct this benchmark to stress-test execution monitoring across varied execution conditions. It combines nominal executions, unsuccessful policy rollouts, and tele-operated trajectories containing execution errors, including runs at different controller update frequencies. This mixture tests the models’ ability to recognize execution phases and distinguish failures from successful behavior across variation in execution timing and rollout quality. Each evaluation query contains the task instruction and an eight-timestep causal observation window from the available two or three camera streams.

For each fine-tuned configuration at each evaluated budget, we report the mean and standard deviation over two SFT seeds. Each run is evaluated using its validation-selected checkpoint, and all methods use the same scoring procedure.

C. Baselines

We compare RoboMonitor with three fine-tuned configurations and one few-shot embodied-reasoning baseline. Downstream SFT settings are closely matched across the fine-tuned models, apart from the supervision and architectural differences described below. RoboMonitor uses a separate frozen visual/context branch and associated fusion and readout modules, which are absent from the Qwen3-VL and Robometer baselines.

Qwen3-VL-4B + Pointwise SFT. We fine-tune Qwen3-VL-4B [22] using the same eight-timestep causal observation windows as the Temporal SFT variant. Only the final readout is supervised. Intermediate readouts receive no supervision, and the within-window and cross-window consistency objectives are omitted. We evaluate this ablation at the 52-episode budget.

Qwen3-VL-4B + Temporal SFT. We initialize from the same Qwen3-VL-4B foundation model and apply Temporal SFT as defined in subsection IV-B. Comparing the Pointwise and Temporal SFT variants measures the combined contribution of dense causal supervision and temporal consistency.

Robometer-4B + Temporal SFT. We initialize from Robometer-4B [14] and apply the same Temporal SFT ob-

jective. This provides a robot-specific reward-model baseline under the same downstream supervision.

Gemini Robotics ER 2 (few-shot). We evaluate Gemini Robotics ER 2 [28] without parameter updates. The model receives labeled in-context examples together with the task instruction and causal camera observations and predicts the same monitoring outputs. We report this result separately from the supervised fine-tuning budgets.

RoboMonitor. RoboMonitor transfers the visual and context encoders learned during predictive pre-training and applies Temporal SFT for downstream execution monitoring.

D. Closed-loop Evaluation

We deploy RoboMonitor in a graph-based task orchestrator similar to GaP [5]. The orchestrator selects low-level skills and uses the monitor’s predictions to track execution progress and trigger predefined recovery behaviors. We evaluate two settings: Toolbox Sorting with a simulated bimanual UR7e system in Isaac Sim, and Reel Packing on a real bimanual UR7e workcell. We conduct 40 trials in each setting.

E. Implementation Details

Predictive pre-training is performed on eight NVIDIA H100 GPUs, and supervised fine-tuning uses four GPUs. The real-world execution monitor runs on a single NVIDIA RTX PRO 6000 Blackwell Max-Q GPU. During deployment, the monitor receives only the task instruction and camera observations; robot state and commands are not required.

F. Metrics

We report phase accuracy and macro recall, task-completion average precision (AP), failure AP, and spurious-switch rate. A spurious switch occurs when the predicted phase changes between adjacent observations while the ground-truth phase remains unchanged; lower is better. For closed-loop evaluation, we report failure-detection recall, false recovery triggers, recovery success, task-completion rate, and inference latency.

VI. RESULTS

A. Label-Efficient Monitoring (Q1)

Table I reports monitoring performance at the two annotation budgets. With 52 labeled episodes, RoboMonitor achieves 93.09% mean phase accuracy and 85.85% macro recall. These exceed Qwen3-VL with Temporal SFT by 6.33 and 11.62 percentage points, respectively, and Robometer with Temporal SFT by 4.32 and 4.07 points. RoboMonitor also achieves the highest task-completion AP at this budget, 87.60%, compared with 85.42% for Robometer and 74.01% for Qwen3-VL.

The cross-budget comparison further supports label efficiency for phase recognition. RoboMonitor trained with 52 episodes exceeds the mean phase accuracy of Robometer trained with 100 episodes (93.09% versus 91.65%), while its macro recall is within 0.37 percentage points (85.85% versus 86.22%). RoboMonitor’s 52-episode model also exceeds Qwen3-VL trained with 100 episodes in phase accuracy (93.09% versus 91.19%) and macro recall (85.85% versus 82.73%). RoboMonitor therefore achieves competitive phase recognition with fewer labeled episodes on this benchmark.

Failure detection shows a different pattern at the smaller budget. With 52 episodes, RoboMonitor obtains 45.87% failure AP, below Robometer’s 59.39% and close to Qwen3-VL’s 46.37%. With 100 episodes, RoboMonitor reaches 93.63% phase accuracy, 88.55% macro recall, 92.29% task-completion AP, and 72.01% failure AP. All four means exceed the corresponding Robometer and Qwen3-VL results. Failure AP is higher by 4.33 and 11.05 percentage points, respectively. The low-label advantage is clearest for phase recognition and completion; stronger failure detection is observed at the larger budget.

Gemini Robotics ER 2 achieves 74.67% phase accuracy and 61.19% macro recall, below all fine-tuned configurations. Its few-shot result provides a reference for monitoring with labeled in-context examples and no parameter updates.

B. Temporal Stability (Q2)

At the 52-episode budget, Qwen3-VL with Temporal SFT reduces mean spurious-switch rate from 15.23% to 4.95% relative to Pointwise SFT, a reduction of 10.28 percentage points. Failure AP increases from 30.34% to 46.37%, while phase accuracy increases more modestly, from 85.42% to 86.76%. Thus, the reduction in switching is accompanied by improved phase accuracy and failure AP. This ablation supports the combined dense-supervision and temporal-consistency recipe.

Across the Temporal SFT configurations, phase accuracy and switching rate favor different methods at the smaller budget. RoboMonitor has the highest phase accuracy with 52 episodes, while its spurious-switch rate is 5.58%, compared with 5.27% for Robometer and 4.95% for Qwen3-VL. With 100 episodes, RoboMonitor achieves the lowest mean switching rate, 3.17%, compared with 3.76% for Robometer and 3.58% for Qwen3-VL. Switching rate should therefore be interpreted alongside phase accuracy when assessing execution-monitoring performance.

TABLE II

CLOSED-LOOP DEPLOYMENT OF ROBOMONITOR. RECOVERY SUCCESS IS MEASURED OVER ATTEMPTED RECOVERIES.

Metric	Reel Packing	Toolbox Sorting
	Real	Sim
Trials	40	40
Failure events	18	12
Failures detected	15/18 (83.3%)	11/12 (91.7%)
False recovery triggers	0	0
Recovery success	13/15 (86.7%)	11/11 (100%)
Task completion	35/40 (87.5%)	39/40 (97.5%)
Inference latency	180–200 ms	180–200 ms

C. Closed-loop Execution and Recovery (Q3)

Table II summarizes deployment within the task orchestrator. In simulated Toolbox Sorting, RoboMonitor detects 11 of 12 failure events across 40 trials. All 11 attempted recoveries succeed, and the integrated system completes 39 trials (97.5%). In real-world Reel Packing, RoboMonitor detects 15 of 18 failure events, and 13 of the 15 attempted recoveries succeed. Thus, 13 of the 18 encountered failures are detected and autonomously recovered (72.2%). The system completes 35 of 40 real-world trials (87.5%). No false recovery triggers are observed in either setting.

Inference takes approximately 180–200 ms per observation window. These trials demonstrate the use of RoboMonitor for task progression and predefined recovery in simulation and on a real robot. These deployments complement the four-task offline stress test by measuring failure-event detection, recovery, and task completion within the integrated monitor-orchestrator system on two tasks.

VII. CONCLUSION

We presented RoboMonitor, a label-efficient execution monitor that combines predictive self-supervised pre-training with Temporal SFT. Across four downstream manipulation tasks, RoboMonitor achieves 93.09% mean phase accuracy and 85.85% macro recall with 52 labeled episodes, exceeding the phase accuracy of the evaluated Qwen3-VL and Robometer baselines trained with 100 episodes. An ablation of the supervised training objective further shows that Temporal SFT reduces the Qwen3-VL spurious-switch rate from 15.23% to 4.95% and increases failure AP from 30.34% to 46.37% relative to Pointwise SFT. In closed-loop deployment, the integrated system completes 39 of 40 simulated Toolbox Sorting trials and 35 of 40 real-world Reel Packing trials, with no false recovery triggers observed in either setting.

A limitation of the current formulation is that it assigns one global execution phase to each timestep. This represents coordinated bimanual behaviors such as handover, but cannot express asynchronous execution in which two manipulators independently perform different subtasks at the same time. Future work could instead use factorized or multi-label execution states for individual manipulators, objects, or concurrent skills. We also plan to augment the monitor’s

categorical outputs with visually grounded language reasoning [29], [30], allowing the planner to receive a description of what changed in the scene in addition to a failure score. Finally, conformal methods [31] could provide calibrated uncertainty for deciding when the system should abstain, request additional observations, or involve a human operator.

REFERENCES

- [1] Haoquan Fang et al. Molmoact2: Action reasoning models for real-world deployment, 2026.
- [2] Kevin Black et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR, 27–30 Sep 2025.
- [3] Moo Jin Kim, Karl Pertsch, et al. Openvla: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 06–09 Nov 2025.
- [4] Brian Ichter et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 14–18 Dec 2023.
- [5] Kaiyuan Chen et al. Gap: A graph-as-policy multi-agent self-learning harness for variational automation tasks, 2026.
- [6] Wenli Xiao et al. Empire: Agentic robot policy self-improvement in the real world, 2026.
- [7] Jiafei Duan et al. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *ArXiv*, abs/2410.00371, 2024.
- [8] Paul Pacaud, Ricardo Garcia, Shizhe Chen, and Cordelia Schmid. Guardian: Detecting robotic planning and execution errors with vision-language models, 2026.
- [9] Christopher Agia et al. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 689–723. PMLR, 06–09 Nov 2025.
- [10] Ralf Römer, Adrian Kobras, Luca Worbis, and Angela P. Schoellig. Failure prediction at runtime for generative robot policies. *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [11] Qiao Gu et al. SAFE: Multitask failure detection for vision-language-action models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [12] Florian Seligmann, Emiliyan Gospodinov, Enes Ulas Dincer, and Gerhard Neumann. Vla-fail: Efficient task failure detection for finetuned vision-language-action models, 2026.
- [13] Wenye Yu, Jun Lv, Zixi Ying, Yang Jin, Chuan Wen, and Cewu Lu. Armada: Autonomous online failure detection and human shared control empower scalable real-world deployment and adaptation, 2025.
- [14] Anthony Liang et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. In *Robotics: Science and Systems 2026*, 2026.
- [15] Qianzhong Chen et al. Sarm2: Multi-task stage aware reward modeling for self improving robotic manipulation. *arXiv preprint arXiv:2606.10305*, 2026.
- [16] Philip Schroeder, Ondrej Biza, Thomas Weng, Hongyin Luo, and James Glass. Rover: Recursive reasoning over videos with vision-language models for embodied tasks, 2025.
- [17] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. In *6th Annual Conference on Robot Learning*, 2022.
- [18] Yecheng Jason Ma et al. VIP: Towards universal visual reward and representation via value-implicit pre-training. In *The Eleventh International Conference on Learning Representations*, 2023.
- [19] Ilija Radosavovic et al. Robot learning with sensorimotor pre-training. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 683–693. PMLR, 06–09 Nov 2023.
- [20] Mahmoud Assran et al. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.
- [21] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.
- [22] Shuai Bai et al. Qwen3-vl technical report, 2025.
- [23] Edward J Hu et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [24] Pulkit Agrawal et al. Learning to poke by poking: Experiential learning of intuitive physics. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.
- [25] Abhiroop Ajith and Constantinos Chamzas. Strips-wm: Learning grounded propositional strips-style world models from images, 2026.
- [26] Dibya Ghosh et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024.
- [27] Yazan Abu Farha and Jurgen Gall. Ms-tcn: Multi-stage temporal convolutional network for action segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.
- [28] Google DeepMind. Gemini robotics ER 2 model card. <https://deepmind.google/models/model-cards/gemini-robotics-er-2/>, 2026. Published July 30, 2026; accessed September 15, 2026.
- [29] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.
- [30] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. *arXiv preprint arXiv:2407.08693*, 2024.
- [31] Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley, Zhenjia Xu, Dorsa Sadigh, Andy Zeng, and Anirudha Majumdar. Robots that ask for help: Uncertainty alignment for large language model planners. In *7th Annual Conference on Robot Learning*, 2023.